POWERGUARD

Smart Electricity Fault Monitoring Platform

Documentation Hub – B.Tech ECE Mini Project

“Monitor Smart. Detect Early. Protect Better.”

Project Domain: Electronics and Communication Engineering

Technology: Web Application + IoT-ready Architecture

Platform: Base44 AI

Live Application: https://vengeful-power-guard-link.base44.app

1. Overview

PowerGuard is a smart electricity monitoring and fault detection platform designed to monitor important electrical parameters, identify abnormal conditions, generate alerts, track equipment health, and support preventive maintenance. The platform is designed as an ECE-focused web application and can later be connected to ESP32-based sensors for real-world IoT monitoring.

2. Abstract

Increasing dependence on electrical and electronic equipment makes continuous monitoring important for safety, reliability and maintenance. Voltage fluctuations, excessive current, overheating, abnormal frequency and unusual power consumption can damage equipment. PowerGuard addresses this problem through a centralized web dashboard that monitors electrical parameters, detects abnormal conditions, generates alerts, maintains fault history and provides equipment-health information. The current demonstration can use simulated sensor data, while the architecture is prepared for future ESP32 and sensor integration.

3. Introduction

Electrical and electronic systems require stable operating conditions. Manual or periodic inspection may not identify faults immediately and often provides limited historical information. PowerGuard combines electronics, embedded-system concepts, IoT communication and web technologies to provide centralized monitoring. It supports early identification of abnormal conditions and provides a structured way to manage equipment, faults, alerts and maintenance.

4. Problem Statement

Electrical and electronic equipment can be damaged due to abnormal voltage, excessive current, overheating, power fluctuations, frequency variations and other electrical faults. Conventional monitoring methods may not provide continuous monitoring, immediate alerts, centralized fault history, or equipment health information. Therefore, a smart platform is required to continuously monitor electrical parameters, identify abnormal conditions and notify users before serious equipment damage occurs.

5. Objectives

1. Monitor important electrical parameters.
2. Detect abnormal electrical conditions.
3. Generate alerts for detected faults.
4. Monitor equipment health.
5. Maintain fault history.
6. Track energy consumption.
7. Provide graphical analytics.
8. Support equipment maintenance.
9. Provide a centralized monitoring dashboard.
10. Prepare the architecture for future IoT sensor integration.

6. Existing System

Traditional approaches commonly depend on manual monitoring, periodic inspection and reactive maintenance. They may provide limited fault history, delayed fault identification, no centralized dashboard, limited analytics and difficult equipment tracking.

7. Proposed System

PowerGuard provides a centralized solution for electrical monitoring and fault management. The proposed architecture is: Electrical Sensors → ESP32/Microcontroller → Wi-Fi Communication → Backend/API → Database → Fault Detection → PowerGuard Dashboard → Alerts and Reports. The current web application can use simulated sensor readings for demonstration, while real sensors can be integrated later.

8. System Architecture

Hardware Layer: voltage sensor, current sensor, temperature sensor, energy/power measurement and ESP32.
Communication Layer: Wi-Fi and HTTP/API or IoT communication.
Backend Layer: data processing, fault detection, alert generation and authentication.
Database Layer: equipment, sensor readings, faults, alerts and maintenance records.
Application Layer: dashboard, live monitoring, equipment, faults, alerts, analytics and reports.

9. Hardware Requirements

ESP32 – main IoT controller.
Voltage Sensor – voltage measurement.
Current Sensor – current measurement.
Temperature Sensor – temperature monitoring.
Energy Sensor – power/energy measurement.
Wi-Fi – data transmission.
Power Supply – system power.

Note: These components represent the proposed/future IoT implementation. The current web application can demonstrate functionality using simulated data.

10. Software Requirements

Base44 AI, web browser, web technologies such as HTML/CSS/JavaScript concepts, database, backend/API services, IoT communication and an ESP32 programming environment for future hardware integration.

11. System Modules

Authentication: user login and access control.
Dashboard: overall system status and electrical statistics.
Live Monitoring: voltage, current, power, frequency and temperature.
Equipment Management: add, edit, view and track equipment.
Fault Detection: identify abnormal electrical conditions.
Alert Management: generate notifications.
Analytics: historical graphs and trends.
Maintenance: maintenance information and scheduling.
Reports: monitoring and fault reports.
Fault Simulation: demonstration of faults without physical hardware.

12. Working Principle

1. Electrical parameters are collected.
2. Sensor/controller provides measurements.
3. Data is transmitted to the backend.
4. Data is stored in the database.
5. Fault detection checks the readings.
6. Normal values are displayed as safe.

7. Abnormal values trigger fault detection.
8. The system generates an alert.
9. Equipment status is updated.
10. Fault information is stored for analysis.
11. Users can acknowledge and resolve faults.
12. Historical information is viewed through analytics.

13. Fault Detection

Over Voltage: voltage exceeds the predefined safe range.
Under Voltage: voltage falls below the safe range.
Over Current: current exceeds equipment rating.
Over Power: power consumption exceeds the configured limit.
High Temperature: temperature exceeds the safe threshold.
Frequency Abnormality: frequency deviates from the operating range.
Power Factor Abnormality: power factor falls below the desired threshold.
Sudden Power Surge: unusual increase in power consumption is detected.

14. Database Design

Users: user_id, name, email, role.
Equipment: equipment_id, name, type, location, voltage_rating, current_rating, power_rating, status, health_score.
Sensor Readings: reading_id, equipment_id, voltage, current, power, frequency, temperature, power_factor, energy, timestamp.
Faults: fault_id, equipment_id, fault_type, severity, detected_value, timestamp, status, resolution.
Alerts: alert_id, fault_id, alert_type, severity, message, timestamp, status.
Maintenance: maintenance_id, equipment_id, maintenance_date, technician, notes, next_maintenance_date.

15. User Interface Documentation

The PowerGuard application includes Login, Dashboard, Live Monitoring, Equipment, Faults, Alerts, Analytics, Reports, Maintenance and Settings pages. The interface is intended to provide clear status indicators, charts, equipment information, fault records and user actions. The live application is available at https://vengeful-power-guard-link.base44.app.

16. Testing

The documentation should cover tests for valid login, equipment addition, over-voltage detection, over-current detection, high-temperature detection, fault resolution, equipment search and report generation. Expected results include successful navigation, database record creation, fault/alert generation and status updates.

17. Results

The developed PowerGuard platform provides centralized monitoring, electrical parameter visualization, fault detection, alert generation, equipment tracking, health monitoring, fault history, analytics and maintenance tracking. The current demonstration uses simulated monitoring data where physical sensors are not connected. Future implementation can use real ESP32 sensor readings.

18. Advantages

Real-time-style monitoring; early fault identification; centralized data; easy equipment tracking; reduced manual monitoring; historical analysis; better maintenance planning; user-friendly dashboard; IoT-ready architecture; scalable design.

19. Limitations

The current version may use simulated sensor data. Physical sensor accuracy depends on hardware and calibration. Internet/Wi-Fi connectivity may be required for remote monitoring. Fault thresholds must be configured according to equipment specifications. Real-world deployment requires electrical safety considerations and suitable protection systems.

20. Future Scope

Future improvements include IoT integration using ESP32 and actual sensors, a mobile application, SMS/email alerts, predictive maintenance, AI-based anomaly detection, energy optimization, cloud deployment, multi-location monitoring and integration with suitable automatic protection hardware.

21. Conclusion

PowerGuard demonstrates how electronics, embedded systems, IoT communication and web technologies can be combined to create a smart electrical monitoring platform. The project moves from conventional reactive monitoring toward smart, centralized and preventive electrical monitoring. With future sensor integration, the platform can become a practical solution for equipment and electrical-condition monitoring.

22. References

ESP32 technical documentation; electrical measurement sensor documentation; IoT communication concepts; power monitoring concepts; embedded systems textbooks; electronics and communication engineering textbooks; web application development documentation. Specific research-paper titles or authors should be added only when actually consulted.

23. Project Team

Student Name: _____
Roll Number: _____
Department: Electronics and Communication Engineering
College: _____
Academic Year: _____
Project Guide: _____

Existing System vs PowerGuard

Feature	Existing System	PowerGuard
Monitoring	Manual/periodic	Centralized monitoring
Fault Detection	Often delayed	Automatic threshold-based detection
Alerts	Manual notification	Dashboard alerts
Data Storage	Limited	Structured database
Analytics	Limited	Charts and historical trends
Equipment Health	Manual assessment	Health score and status
Maintenance	Reactive	Maintenance tracking